

SIF x OpenClaw Integration Architecture

Amendment A31: Sovereign Intelligence Runtime Layer

Version: 1.0.0
Status: DRAFT
Author: Daniel Shamir / SIF Architecture
Date: 2026-02-20
Classification: SIF Amendment Series

1. Executive Summary

This amendment defines the integration architecture between the Sovereign Intelligence Framework (SIF) and OpenClaw, the open-source personal AI agent platform (145K+ GitHub stars). The thesis is simple: **SIF is the Soul, OpenClaw is the Body.**

SIF provides the persistent intelligence layer — the pointer graph, archetypes, Hebbian reinforcement, provider sovereignty, and accumulated wisdom. OpenClaw provides the execution runtime — the always-on gateway, multi-channel messaging, tool sandboxing, skill execution, and browser/system access.

Together they create something neither can achieve alone: **an autonomous, sovereign AI agent that remembers everything, acts on your behalf, runs on your hardware, and whose accumulated intelligence you own outright.**

2. Philosophical Alignment

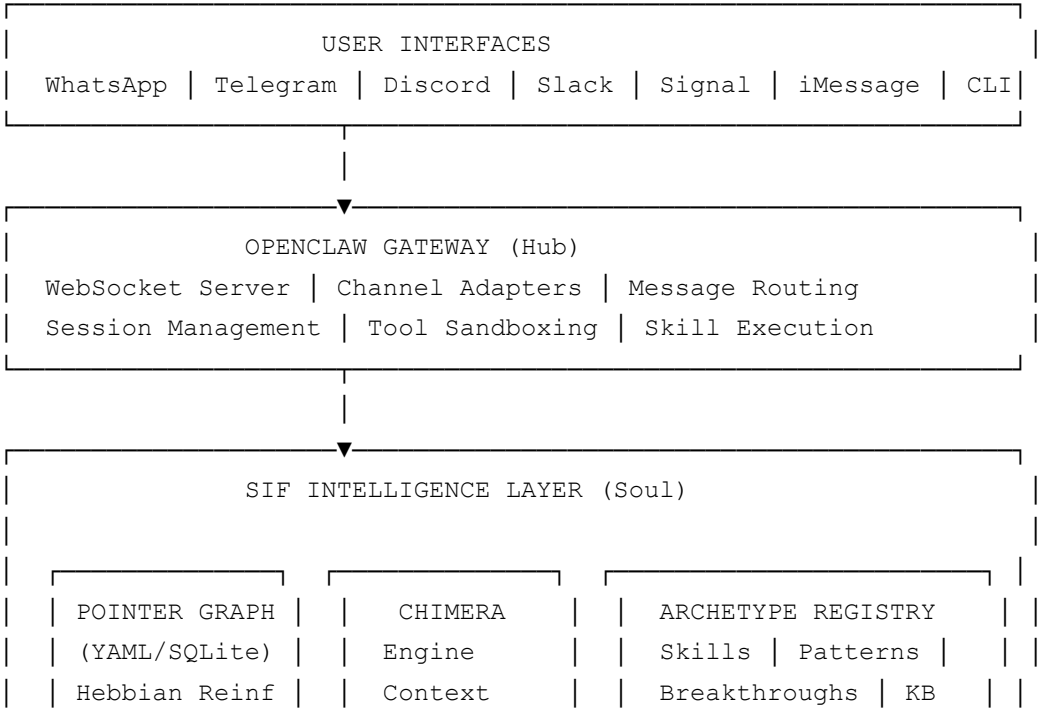
Principle	SIF	OpenClaw	Synergy
Sovereignty	Intelligence lives in user-owned pointer graph	Context/skills live on user’s machine	Full stack sovereignty: both intelligence AND execution
Provider Agnostic	LLM is the CPU — switch providers, keep wisdom	Supports Anthropic, OpenAI, local	SIF archetypes persist across ANY model OpenClaw routes to

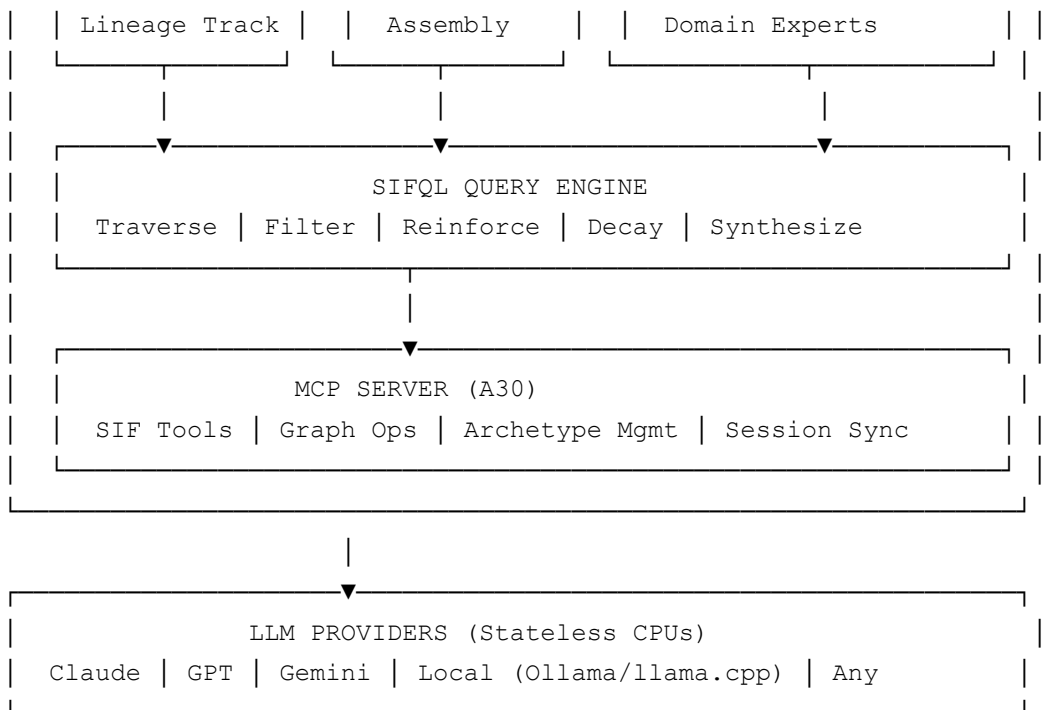
		models	
Open Architecture	Amendments are public specifications	MIT-licensed, community-driven	Both ecosystems benefit from open extension
Anti-Lock-in	Escape hatch from AI memory lock-in	Escape from walled-garden assistants	Complete liberation from vendor ecosystems
Persistence	Pointer graph survives sessions, providers, time	Memory files survive context compaction	Layered persistence: operational (OC) + strategic (SIF)

The Fundamental Insight

OpenClaw’s memory architecture (flat Markdown files + hybrid search) is its **weakest layer** — multiple community projects (hippocampus-memory, graph-memory, Graphiti integration, 12-layer memory architecture) are all attempts to solve what SIF already solves. Meanwhile, SIF lacks an **execution runtime** — it has no hands, no eyes, no autonomous action capability. This integration fills both gaps simultaneously.

3. Architecture Overview





4. Component Mapping: Every SIF Element → OpenClaw Integration

4.1 Pointer Graph → OpenClaw Memory Plugin

SIF Component: The pointer graph (YAML/SQLite) is the persistent intelligence substrate — nodes represent knowledge, edges represent relationships, weights encode Hebbian reinforcement, and lineage chains track how intelligence was acquired.

OpenClaw's Current State: Flat Markdown files (`MEMORY.md` + `memory/YYYY-MM-DD.md`) with hybrid BM25/vector search. No graph structure, no reinforcement, no lineage.

Integration Design:

```
# OpenClaw plugin configuration
plugins:
  slots:
    memory: "sif-memory" # Replaces default memory-core
  entries:
    sif-memory:
      enabled: true
      config:
        graphPath: "~/sif/pointer-graph.yaml"
        dbPath: "~/sif/pointer-graph.sqlite"
```

```

hebbianDecay: 0.95          # Daily decay factor
reinforcementThreshold: 3    # Min activations to promote
lineageTracking: true
exportMarkdown: true        # Also write MEMORY.md for OC compatibility

```

Bridge Mechanics:

- `memory_search` → SIFQL traversal with semantic ranking (pointer graph nodes ranked by activation weight × semantic similarity × recency)
- `memory_get` → Direct pointer graph node retrieval with full lineage
- Write operations → Pointer creation/reinforcement in graph + Markdown shadow for backward compatibility
- OpenClaw's pre-compaction memory flush → SIF session checkpoint (writes active pointers to graph before context compaction)
- OpenClaw's temporal decay → Maps to SIF's Hebbian decay but with graph-aware decay (connected nodes decay slower)

What This Gives OpenClaw Users:

- Intelligence that transcends individual sessions
- Accumulated patterns, not just facts
- Full provenance on every piece of knowledge
- Survive provider switches without losing anything

4.2 CHIMERA → OpenClaw System Prompt + Context Assembly

SIF Component: CHIMERA (Claude Hybrid Intelligence Memory Engine for Reasoning Amplification) is the context assembly engine that determines which pointers, archetypes, and patterns to inject into the LLM's context window for any given interaction.

OpenClaw's Current State: Static system prompt (`SOUL.md`) + recent memory files loaded at session start. No dynamic context optimization.

Integration Design:

```

// CHIMERA as OpenClaw context middleware
interface ChimeraContextProvider {
    // Called by OpenClaw's agent runtime before each LLM call
    assembleContext(params: {

```

```

    sessionHistory: Message[];
    activePointers: Pointer[];
    query: string;
    availableTokens: number;
    channelType: 'direct' | 'group' | 'broadcast';
  }): ContextPayload;
}

```

CHIMERA Context Assembly Pipeline:

1. **Intent Classification** — Analyze user message to determine domain, urgency, depth
2. **Pointer Activation** — Fire spreading activation through graph from intent keywords
3. **Archetype Selection** — Pick relevant archetypes based on activated subgraph
4. **Token Budget** — Allocate context window: system prompt (fixed) → archetypes (priority) → active pointers (ranked) → session history (remainder)
5. **Context Injection** — Format selected intelligence as structured context block for OpenClaw's system prompt prepend

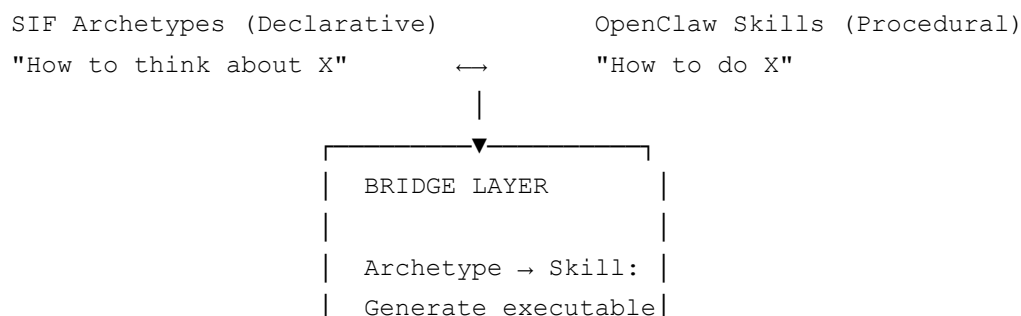
Key Innovation: OpenClaw currently loads the same MEMORY.md every session. CHIMERA dynamically selects the *right* intelligence for the *right* moment based on graph topology, not just keyword matching.

4.3 Archetypes & Skills → OpenClaw Skills Bidirectional Bridge

SIF Component: Archetypes are declarative intelligence patterns extracted from problem-solving sessions — reusable templates for how to think about recurring challenges. Skills are procedural capabilities derived from demonstrated competence.

OpenClaw's Current State: Skills are self-contained executable scripts (bash/Python/Node) that the agent can discover, install, and even write itself. Procedural only.

Bidirectional Bridge Design:



skill from pattern
Skill → Archetype:
Extract pattern
from usage logs

Archetype → Skill Translation Example:

```
# SIF Archetype (declarative pattern)
archetype:
  name: "diagnostic-triage"
  domain: "healthcare/kimera"
  pattern: |
    When a Kimera unit reports anomalous readings:
    1. Check calibration status (last calibration > 24h = suspect)
    2. Compare against control baseline
    3. If delta > 2 standard deviations, flag for manual review
    4. Log all decisions with timestamp for audit trail
  learned_from: ["session-2025-11-15", "session-2025-12-03"]
  reinforcement_count: 7

#!/bin/bash
# Auto-generated OpenClaw skill from SIF archetype: diagnostic-triage
# Generated: 2026-02-20 | Source: pointer-graph://archetypes/diagnostic-triage

DESCRIPTION="Triage Kimera diagnostic anomalies using the established protocol"
ARGUMENTS="unit_id:string calibration_window_hours:number=24"

# [Generated procedural implementation of the declarative pattern]
# ... executable code that implements the archetype ...
```

Skill → Archetype Extraction: When OpenClaw executes a skill repeatedly with consistent patterns, the bridge extracts the underlying pattern and creates/reinforces a SIF archetype. This is **Hebbian learning through operational execution** — the agent learns by doing.

4.4 SIFQL → OpenClaw Memory Search Enhancement

SIF Component: SIFQL is the query language for traversing the pointer graph — supporting traversal, filtering, reinforcement, decay, and synthesis operations.

Integration as Enhanced memory_search:

```
# OpenClaw memory_search with SIFQL backend
memory_search("how did we solve the authentication problem?")

# Under the hood, translates to:
TRAVERSE FROM intent("authentication problem solving")
  FOLLOW edges(type="solution", "implementation", "decision")
  FILTER weight > 0.3
  REINFORCE matched_nodes BY 0.1
  RETURN TOP 5 BY activation_score
  WITH LINEAGE depth=2
```

Extended Tool Surface:

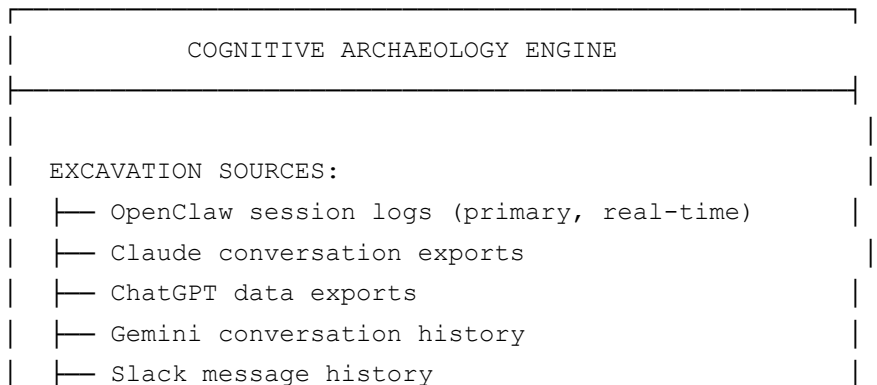
```
// New tools exposed to OpenClaw agent
tools: [
  "sif_traverse",      // Graph traversal queries
  "sif_reinforce",     // Explicit reinforcement of patterns
  "sif_create_pointer", // Create new knowledge nodes
  "sif_link",          // Create edges between nodes
  "sif_archetype_match", // Find matching archetypes for current context
  "sif_lineage",       // Trace provenance of any knowledge
]
```

4.5 Cognitive Archaeology → OpenClaw Session Import Pipeline

SIF Component: Cognitive Archaeology protocols for importing conversation history from multiple AI platforms (Claude, ChatGPT, Gemini, etc.) to bootstrap SIF instances with pre-learned user understanding.

OpenClaw Enhancement: OpenClaw already captures sessions across WhatsApp, Telegram, Discord, Slack, Signal, iMessage. The Cognitive Archaeology pipeline can ingest ALL of these channels as excavation sources.

Pipeline Architecture:



	└─ Email threads (via OpenClaw Gmail skill)	
	EXTRACTION LAYERS:	
	└─ Entity extraction (people, projects, orgs)	
	└─ Decision extraction (choices, rationale)	
	└─ Pattern extraction (recurring approaches)	
	└─ Preference extraction (style, tools, methods)	
	└─ Relationship mapping (who works with whom)	
	└─ Timeline construction (project evolution)	
	OUTPUT → Pointer Graph Bootstrap	
	└─ Initial nodes with source attribution	
	└─ Edge weights from co-occurrence frequency	
	└─ Archetype seeds from pattern clusters	
	└─ Confidence scores per extraction	

Real-Time vs. Historical:

- **Historical:** One-time bulk import of existing conversation archives
- **Real-Time:** OpenClaw sessions continuously feed Cognitive Archaeology for ongoing intelligence extraction

4.6 Replicator Layer / Digital Twins → OpenClaw Multi-Agent

SIF Component: The Replicator Layer enables digital twins — portable SIF instances that can be deployed, forked, and specialized for different domains.

OpenClaw's Multi-Agent: OpenClaw supports multiple agent configurations with different SOULs, session keys, and memory spaces. Agents can spawn sub-agents programmatically.

Integration Design:

```
# Multi-agent SIF deployment on OpenClaw
agents:
  main:
    soul: "SOUL.md"
    sif:
      graphPath: "~/sif/graphs/daniel-primary.yaml"
      mode: "full" # Full read/write access

  kimera-ops:
    soul: "SOUL-kimera.md"
```



```

sif:
  graphPath: "~/sif/graphs/kimera-specialist.yaml"
  parentGraph: "~/sif/graphs/daniel-primary.yaml"
  mode: "domain-filtered" # Only healthcare/diagnostic pointers
  replicatorSync: "bidirectional" # Learnings flow back to parent

abacus-dev:
  soul: "SOUL-abacus.md"
  sif:
    graphPath: "~/sif/graphs/abacus-specialist.yaml"
    parentGraph: "~/sif/graphs/daniel-primary.yaml"
    mode: "domain-filtered" # Only avatar/platform pointers

gabriel-research:
  soul: "SOUL-research.md"
  sif:
    graphPath: "~/sif/graphs/research-track.yaml"
    parentGraph: "~/sif/graphs/daniel-primary.yaml"
    mode: "read-fork" # Can read parent, writes to own graph

```

Replicator Sync Protocol:

- Parent graph pointers are readable by child agents (filtered by domain tags)
- Child agent discoveries create local pointers first
- High-confidence discoveries (reinforcement > threshold) propagate back to parent
- Conflict resolution: parent graph is canonical, child suggestions are flagged for review

4.7 Genesis Bootstrapping → OpenClaw Onboarding

SIF Component: Genesis project bootstrapping protocol for initializing new projects with SIF context — generates CLAUDE.md, progress tracking, Docker configs.

OpenClaw Enhancement: Genesis becomes the “smart onboarding” for new OpenClaw + SIF installations.

```

# Enhanced OpenClaw onboarding with Genesis
openclaw onboard --with-sif

# Genesis protocol activates:
# 1. Interactive requirements discovery (existing SIF skill)
# 2. Pointer graph initialization from conversation
# 3. Archetype seeding based on detected user patterns
# 4. Domain detection → specialist agent recommendations

```

```
# 5. CLAUDE.md generation tuned for OpenClaw runtime
# 6. Cognitive Archaeology import from existing AI platforms
```

4.8 MGMO Orchestration → OpenClaw Multi-Agent Routing

SIF Component: Master Guru / Minion Orchestration Protocol — decomposes complex tasks into specialist minion analyses across multiple dimensions.

OpenClaw Mapping: MGMO minions map to OpenClaw sub-agents with specialized SIF subgraphs.

```
// MGMO orchestration via OpenClaw sub-agents
async function masterGuruOrchestrate(task: ComplexTask) {
  const minions = await sif.archetype_match(task, { type: "specialist" });

  const results = await Promise.all(
    minions.map(minion =>
      openclaw.sessions_spawn({
        agent: minion.agentId,
        sifSubgraph: minion.domainFilter,
        prompt: minion.analysisPrompt,
        timeout: 120_000,
        returnFormat: "structured"
      })
    )
  );

  return synthesize(results); // Master Guru synthesizes minion outputs
}
```

4.9 Provider Sovereignty → OpenClaw Model Routing

SIF Component: The ability to switch between any LLM provider while preserving all accumulated intelligence.

OpenClaw Enhancement: OpenClaw already supports multi-provider routing. Combined with SIF, the pointer graph ensures:

- Switch from Claude to GPT → All archetypes, patterns, knowledge persist
- Switch to local model → Same intelligence, private execution
- A/B test providers → Same SIF context, different LLM reasoning
- Fallback chains → If provider is down, intelligence travels with the request

```
# Provider-sovereign configuration
models:
  primary:
    provider: "anthropic"
    model: "claude-sonnet-4-5-20250929"
  fallback:
    - provider: "openai"
      model: "gpt-4o"
    - provider: "local"
      model: "qwen3:32b"

# SIF ensures the SAME intelligence context regardless of which model processes i
sif:
  providerSovereignty:
    contextFormat: "universal" # Normalize context for any provider
    archetype_encoding: "provider-agnostic" # No provider-specific tokens
```

4.10 Session Management → OpenClaw Session Lifecycle

SIF Component: Session logging (`session-log.md`) with pointer graph checkpoints.

OpenClaw Mapping: OpenClaw has sophisticated session management with compaction, pruning, and transcript indexing.

Enhanced Lifecycle:

Session Start

- |— OpenClaw creates/resumes session
- |— SIF loads active pointer subgraph
- |— CHIMERA assembles initial context
- └─ Session-log checkpoint: "session-opened"

During Session

- |— Each exchange → SIF reinforces activated pointers
- |— New knowledge → SIF creates pointers with OC session lineage
- |— Archetype matches → Logged in session-log
- └─ Periodic → SIF graph snapshot (every N exchanges)

Pre-Compaction (OpenClaw's memory flush)

- |— SIF checkpoint: write all active pointers to graph
- |— Extract session archetypes → pointer graph
- |— Session-log entry: "pre-compaction-flush"
- └─ OC compacts context → SIF intelligence survives

Session End

- |— Final SIF checkpoint

- └ Cognitive Archaeology extraction on session transcript
- └ Reinforcement summary → which pointers strengthened
- └ Session-log: "session-closed" with delta report

4.11 Heartbeat System → Proactive Intelligence

OpenClaw Feature: Heartbeat — periodic background check-ins where the agent proactively reaches out.

SIF Enhancement: Heartbeats become intelligence-driven, not just schedule-driven.

```
# SIF-aware heartbeat configuration
agents:
  defaults:
    heartbeat:
      interval: "4h"
    sif:
      # Trigger heartbeat when pointer graph detects:
      triggers:
        - type: "decaying-critical" # Important pointer about to decay below t
          action: "remind-user"
        - type: "pattern-complete" # Enough data to form new archetype
          action: "notify-discovery"
        - type: "conflict-detected" # Contradictory pointers found
          action: "request-resolution"
        - type: "project-stale" # Active project pointers not reinforced i
          action: "check-in"
```

4.12 AHCA (Avatar Hypervisor) → OpenClaw Channel Architecture

SIF Component: Avatar Hypervisor Compute Architecture — distributed operating system with hierarchical layers for avatar interfaces.

OpenClaw as AHCA Runtime:

```
AHCA Layer 4: Applications    → OpenClaw Skills + Abacus Avatar UI
AHCA Layer 3: Services        → OpenClaw Gateway + SIF MCP Server
AHCA Layer 2: Orchestration   → MGMO via OpenClaw Multi-Agent
AHCA Layer 1: Infrastructure   → OpenClaw on Mac Mini / VPS / Cloud
```

Abacus Avatars can be deployed as an OpenClaw channel adapter — users interact with their SIF-powered intelligence through avatar interfaces while the execution happens through OpenClaw's runtime.

4.13 Nexless/Kimera Integration → Domain-Specific Agent

SIF Component: Kimera P-IV RT-qPCR diagnostic platform operational intelligence.

OpenClaw Deployment:

```
# Kimera operations agent
agents:
  kimera-ops:
    soul: "SOUL-kimera-ops.md"
    sif:
      graphPath: "~/sif/graphs/kimera-operations.yaml"
      domainFilter: ["healthcare", "diagnostics", "kimera", "qpcr"]
    channels:
      - telegram # Field technician alerts
      - slack     # Operations team channel
    skills:
      - kimera-unit-health-check
      - kimera-calibration-reminder
      - kimera-anomaly-triage      # Generated from SIF archetype
      - kimera-batch-report
    heartbeat:
      interval: "6h"
      check: "kimera-fleet-status"
```

5. Technical Implementation Phases

Phase 1: SIF Memory Plugin (Weeks 1-4)

Goal: Replace OpenClaw's default memory with SIF pointer graph backend.

Deliverables:

- OpenClaw memory plugin (`sif-memory`) implementing `MemoryProvider` interface
- Pointer graph ↔ Markdown bidirectional sync (backward compatibility)
- `memory_search` backed by SIFQL traversal + hybrid BM25/vector
- `memory_get` with full pointer lineage
- Pre-compaction flush → SIF checkpoint
- NPX-distributable via A30 MCP architecture

Key Files:

```

~/openclaw/extensions/sif-memory/
├─ package.json          # OpenClaw plugin manifest
├─ src/
│   ├─ provider.ts       # MemoryProvider implementation
│   ├─ graph-adapter.ts  # Pointer graph read/write
│   ├─ sifql-search.ts   # SIFQL-backed search
│   ├─ hebbian.ts        # Reinforcement/decay engine
│   ├─ markdown-sync.ts  # Bidirectional MD shadow
│   └─ compaction-hook.ts # Pre-compaction checkpoint
├─ SKILL.md              # OpenClaw skill definition
└─ tests/

```

Phase 2: CHIMERA Context Engine (Weeks 5-8)

Goal: Dynamic context assembly replacing static MEMORY.md loading.

Deliverables:

- CHIMERA context middleware for OpenClaw agent runtime
- Intent classification → pointer activation pipeline
- Token-budget-aware context selection
- Archetype injection into system prompt
- Channel-aware context (DM vs group vs broadcast)

Phase 3: Archetype ↔ Skill Bridge (Weeks 9-12)

Goal: Bidirectional translation between SIF archetypes and OpenClaw skills.

Deliverables:

- Archetype → Skill code generator
- Skill execution → Archetype extraction pipeline
- Skill marketplace integration (ClawHub + SIF archetype tagging)
- Auto-discovery of new archetypes from operational patterns

Phase 4: Cognitive Archaeology Pipeline (Weeks 13-16)

Goal: Import intelligence from all historical AI conversations.

Deliverables:

- OpenClaw session → SIF extraction (real-time)

- Claude/ChatGPT/Gemini export parsers
- Entity/decision/pattern extraction engine
- Pointer graph bootstrap from archaeological data
- Confidence scoring and human review interface

Phase 5: Full Ecosystem Integration (Weeks 17-20)

Goal: MGMO, Replicator, AHCA, and domain agents.

Deliverables:

- MGMO → OpenClaw multi-agent orchestration
 - Replicator → Agent forking with filtered SIF subgraphs
 - AHCA → Abacus Avatar as OpenClaw channel
 - Kimera Operations agent deployment
 - Genesis-enhanced onboarding flow
-

6. Distribution Strategy

For SIF:

- OpenClaw's 145K+ star community becomes distribution channel
- Every SIF-memory plugin install = new SIF user
- ClawHub (OpenClaw's skill marketplace) hosts SIF skills
- SIF becomes "the intelligence layer every OpenClaw user wants"

For OpenClaw:

- SIF solves their biggest weakness (flat memory)
- Enterprise users get sovereign intelligence + audit trails
- Healthcare vertical (Kimera) demonstrates regulated-industry capability
- Academic track (Gabriel's research) provides legitimacy

Positioning:

- Not competing with OpenClaw — **enhancing** it
 - SIF is the “upgrade your OpenClaw’s brain” plugin
 - OpenClaw is “give your SIF a body”
 - Together: the first truly sovereign personal AI infrastructure
-

7. Security & Sovereignty Considerations

Data Flow Boundaries

- Pointer graph NEVER leaves user’s machine unless explicitly federated
- OpenClaw’s tool sandboxing applies to SIF operations
- MCP server runs local-only by default (no cloud dependency)
- Lineage tracking provides full audit trail for regulated industries (HIPAA for Kimera)

Provider Isolation

- SIF context is formatted provider-agnostic (no Anthropic-specific tokens)
 - Pointer graph encryption at rest (SQLite encryption extension)
 - Per-session ephemeral keys for MCP communication
 - Credential isolation: SIF MCP server never touches OpenClaw API keys
-

8. Relationship to Existing Amendments

Amendment	Relationship to A31
A1 (Version Control)	Git-based pointer graph versioning through OpenClaw workspace
A7 (CHIMERA)	CHIMERA becomes OpenClaw context middleware (§4.2)
A10 (SIFQL)	SIFQL backs OpenClaw memory_search tool (§4.4)
A15 (Cognitive Archaeology)	OpenClaw sessions become primary excavation source (§4.5)

A20 (Replicator Layer)	Maps to OpenClaw multi-agent architecture (§4.6)
A25 (MCP Distribution)	NPX distribution targets OpenClaw plugin ecosystem (§5 Phase 1)
A30 (MCP Architecture)	MCP server is the bridge protocol between SIF and OpenClaw (§3)

9. Success Metrics

1. **Plugin Adoption:** 1,000+ OpenClaw installations with SIF-memory plugin within 6 months
 2. **Intelligence Persistence:** Users retain 95%+ of accumulated intelligence across provider switches
 3. **Operational Autonomy:** SIF-powered OpenClaw agents executing 50+ autonomous actions/day
 4. **Community Contribution:** 10+ community-built SIF archetypes published to ClawHub
 5. **Enterprise Validation:** 1 regulated-industry deployment (Kimera) passing compliance audit
-

10. Open Questions

1. **Licensing:** SIF plugin for OpenClaw — MIT to match OC, or dual-license with commercial tier?
 2. **Graph Format:** YAML (human-readable, SIF native) vs SQLite-only (performance) vs hybrid?
 3. **Federation:** When/how do SIF instances share intelligence across users (enterprise use case)?
 4. **Gabriel's Research Track:** Can the academic work formalize the Hebbian pointer graph as publishable contribution?
 5. **Naming:** "SIF Memory" plugin? "Sovereign Memory"? "Soul Plugin"? Community naming?
-

"The LLM is the CPU, the Pointer Graph is the Soul, and now the Soul has a Body."

Next Steps:

- Push to SIF-Knowledge-Base as `amendments/A31-OpenClaw-Integration/`
- Create GitHub issue on `openclaw/openclaw` for plugin architecture discussion
- Prototype `sif-memory` plugin (Phase 1, minimal viable)
- Engage OpenClaw community (Discord) for early feedback
- Coordinate with Gabriel on academic implications